ANALIZADOR LÉXICO DE FLOWXYRUST

Especificación del analizador léxico

Integrantes: Sotelo Atuncar, Diego Alejandro
Rodriguez Claeyssen, Isaac Aaron
Emilio Denis, Cabello Villon

1. Definición del léxico y tokens

El analizador léxico recorre el programa carácter por carácter, forma lexemas y asigna un token a cada componente reconocido.

Las unidades léxicas consideradas en FlowxyRust son:

- palabras reservadas;
- identificadores;
- literales numéricos;
- literales de carácter;
- literales de cadena;
- operadores;
- delimitadores;
- comentarios.

Los espacios, tabulaciones, saltos de línea y comentarios se reconocen durante el recorrido, pero se descartan.

1.1. Palabras reservadas

N.	Lexema	Token
1	`sea`	100
2	`constante`	101
3	`mutable`	102
4	`clase`	103
5	`implementa`	104
6	`este`	105
7	`entero16`	106
8	`entero32`	107
9	`entero64`	108
10	`natural16`	109
11	`natural32`	110
12	`natural64`	111

N.	Lexema	Token
13	`flotante16`	112
14	`flotante32`	113
15	`flotante64`	114
16	`caracter`	115
17	`cadena`	116
18	`booleano`	117
19	`verdadero`	118
20	`falso`	119
21	`leer`	120
22	`mostrar`	121
23	`si`	122
24	`sino`	123
25	`mientras`	124
26	`para`	125
27	`fun`	126
28	`retornar`	127
29	`mainfun`	128

1.2. Operadores

Cada operador tiene su propio token:

N.	Operador	Token
1	-	200
2	!	201
3	+	202
4	*	203
5	/	204
6	%	205
7	<	206
8	<=	207
9	>	208
10	>=	209
11	==	210
12	!=	211
13	&&	212
14	||	213
15	=	214
16	+=	215
17	-=	216
18	*=	217
19	/=	218
20	%=	219
21	++	220
22	--	221

Cuando varios operadores comparten el mismo primer carácter, se revisa el carácter siguiente antes de cerrar el lexema.

1.3. Delimitadores

Lexema	Descripción	Token
(	Delimitador	300
)	Delimitador	301
{	Delimitador	302
}	Delimitador	303
[	Delimitador	304
]	Delimitador	305
:	Delimitador	306
;	Delimitador	307
.	Delimitador	308
,	Delimitador	309

1.4. Otras unidades léxicas

Unidad léxica	Token
Identificador	1000
Literal numérico	2000
Literal de carácter	3000
Literal de cadena	4000

Los enteros y decimales se reconocen mediante caminos distintos del autómata, pero en esta versión ambos utilizan el token general `2000`.

2. Especificación del analizador léxico

La especificación se realiza mediante patrones, expresiones regulares, gramática regular y autómatas.

2.1. Patrones léxicos

Un patrón indica cómo se forma una unidad léxica carácter por carácter.

Se utilizará:

$$L = [A - Za - z] \qquad d = [0 - 9]$$

Unidad	Inicio	Continuación y final
Identificador	Una letra	Continúa con letras, dígitos o _. El _ debe ir seguido de letra o dígito.
Literal numérico	Un dígito	Continúa con dígitos. Puede aparecer un punto seguido de uno o más dígitos.
Literal de cadena	Comilla doble	Continúa con caracteres permitidos hasta encontrar otra comilla doble.
Literal de carácter	Comilla simple	Se lee exactamente un carácter y luego debe aparecer la comilla simple de cierre.
Comentario	`//`	Continúa hasta el salto de línea y luego se descarta.
Palabra reservada	Una letra	Primero se reconoce como palabra con forma de identificador y después se compara con la tabla de reservadas.
Operador	Símbolo de operador	Se revisa el siguiente carácter si puede formar un operador de dos caracteres.
Delimitador	Símbolo delimitador	Se reconoce directamente como un lexema de un carácter.

Ejemplos carácter por carácter

Identificador:

```
numero_total
```

$n \rightarrow$ letra $\rightarrow u, m, e, r, o \rightarrow$ continúa $\rightarrow$ _ $\rightarrow$ espera letra/dígito $\rightarrow total \rightarrow$ continúa

Resultado:

`numero_total` $\rightarrow$ Identificador $\rightarrow 1000$

Literal numérico:

```
15.5
```

$1, 5 \rightarrow$ dígitos $\rightarrow$. $\rightarrow$ posible decimal $\rightarrow 5 \rightarrow$ decimal válido

Resultado:

`15.5` $\rightarrow$ Literal numérico $\rightarrow 2000$

Cadena:

```
"Hola"
```

" $\rightarrow$ inicio $\rightarrow H, o, l, a \rightarrow$ contenido $\rightarrow$ " $\rightarrow$ fin

Carácter:

```
'D'
```

' $\rightarrow$ inicio $\rightarrow D \rightarrow$ carácter $\rightarrow$'$\rightarrow$ fin

2.2. Expresiones regulares

Las expresiones regulares utilizadas son:

Identificador	:	$[A-Za-z]([A-Za-z0-9] \mid _[A-Za-z0-9])^*$
Entero	:	d^+
Decimal	:	$d^+.d^+$
Literal numérico	:	$d^+ \mid d^+.d^+$
Cadena	:	"C^*"
Carácter	:	'C'
Comentario	:	//C^*

Los operadores y delimitadores corresponden a los lexemas exactos definidos en las tablas de la sección anterior.

2.3. Gramática regular

La gramática regular se representa como:

$$G = (V_n, V_t, P, S)$$

donde:

Símbolo	Significado
G	Gramática regular
V_n	Conjunto de símbolos no terminales
V_t	Conjunto de símbolos terminales
P	Conjunto de producciones
S	Símbolo inicial

Para identificadores y literales numéricos:

$$V_n = \{S, I, U, N, P, D\}$$

Los no terminales representan:

Símbolo	Representación
S	Inicio del reconocimiento. Desde aquí puede comenzar un identificador o un número.
I	Identificador en un estado válido; puede continuar con letra, dígito o _.
U	Se acaba de leer _; se espera una letra o un dígito para continuar el identificador.
N	Parte entera de un literal numérico.
P	Se acaba de leer el punto decimal; se espera un dígito.
D	Parte decimal del literal numérico.

Aquí la letra P dentro de V_n representa el estado posterior al punto decimal; no debe confundirse con P, que en $G = (V_n, V_t, P, S)$ representa el conjunto de producciones.

$$V_t = \{L, d, _, .\}$$

donde:

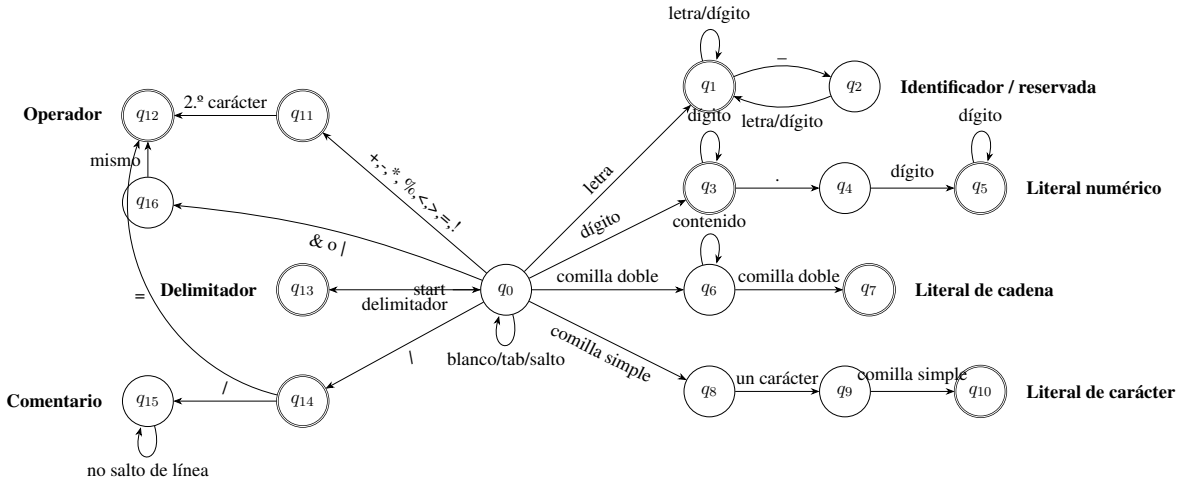
Símbolo	Significado
L	Una letra de `A-Z` o `a-z`
d	Un dígito de `0-9`
_	Guion bajo
.	Punto decimal
λ	Cadena vacía; permite finalizar el lexema en un estado válido

$$P = \begin{cases} S \to LI \mid dN \\ I \to LI \mid dI \mid _U \mid \lambda \\ U \to LI \mid dI \\ N \to dN \mid .P \mid \lambda \\ P \to dD \\ D \to dD \mid \lambda \end{cases}$$

Estas producciones representan la formación de identificadores y literales numéricos carácter por carácter.

2.4. Autómata del analizador léxico

El autómata se presenta de forma resumida. Desde el estado inicial q_0 se selecciona el camino según el primer carácter leído. Los estados con doble círculo indican que el lexema ya puede considerarse reconocido.



Significado de los estados:

Estado	Significado	Estado	Significado
q_0	Estado inicial	q_8	Inicio del literal de carácter
q_1	Identificador o reservada reconocida	q_9	Carácter leído
q_2	Después de _; espera letra o dígito	q_{10}	Literal de carácter reconocido
q_3	Literal numérico entero reconocido	q_{11}	Operador simple reconocido
q_4	Punto leído; espera un dígito	q_{12}	Operador compuesto reconocido
q_5	Literal numérico decimal reconocido	q_{13}	Delimitador reconocido
q_6	Leyendo el contenido de una cadena	q_{14}	Operador / o inicio de /= / //
q_7	Literal de cadena reconocido	q_{15}	Comentario de una línea; se descarta
q_{16}	Se leyó & o |; espera el mismo carácter		

La rama de operadores resume los operadores definidos para FlowxyRust. Por ejemplo:

$$>,\ >= \qquad +,\ +=,\ ++ \qquad -,\ -=,\ --$$

El carácter / se muestra aparte porque puede producir tres resultados:

Primer carácter	Siguiente	Resultado
/	=	operador /=
/	/	comentario de una línea
/	otro	operador /

2.5. Tabla de transición

La tabla resume los movimientos principales del autómata:

Estado	Entrada	Siguiente	Resultado
q_0	letra	q_1	iniciar identificador/reservada
q_1	letra o dígito	q_1	continuar identificador
q_1	_	q_2	esperar letra o dígito
q_2	letra o dígito	q_1	volver a identificador válido
q_0	dígito	q_3	iniciar número
q_3	dígito	q_3	continuar entero
q_3	.	q_4	posible decimal
q_4	dígito	q_5	confirmar decimal
q_5	dígito	q_5	continuar decimal
q_0	comilla doble	q_6	iniciar cadena
q_6	carácter permitido	q_6	continuar cadena
q_6	comilla doble	q_7	finalizar cadena
q_0	comilla simple	q_8	iniciar carácter
q_8	un carácter	q_9	guardar carácter
q_9	comilla simple	q_{10}	finalizar carácter
q_0	operador simple excepto /	q_{11}	reconocer operador simple
q_{11}	continuación válida	q_{12}	reconocer operador compuesto
q_0	& o |	q_{16}	esperar segundo carácter
q_{16}	mismo carácter	q_{12}	reconocer && o ||
q_0	/	q_{14}	reconocer / o esperar continuación
q_{14}	=	q_{12}	reconocer /=
q_{14}	/	q_{15}	iniciar comentario
q_{15}	no salto de línea	q_{15}	continuar comentario
q_0	delimitador	q_{13}	reconocer delimitador
q_0	blanco/tab/salto	q_0	ignorar y continuar

Los estados con doble círculo son estados de aceptación. Los estados q_2, q_4, q_6, q_8, q_9, q_{15} y q_{16} necesitan seguir leyendo antes de cerrar el reconocimiento.

Caso del carácter `/`: el lexema `/=` no es un solo carácter, sino un operador compuesto por los caracteres `/` y `=`. Si después de `/` aparece otro `/`, comienza un comentario `//`. En cualquier otro caso, `/` se reconoce como operador de división. Por ejemplo, en `A/B` el lexema central es `/`.

3. Errores léxicos básicos

El analizador léxico solo puede detectar errores relacionados directamente con la formación de un lexema.

Se consideran casos básicos:

- un carácter que no pertenece a ningún patrón, por ejemplo `@`;
- un identificador que queda incompleto después de `_`, por ejemplo `edad_`;
- dos guiones bajos consecutivos, por ejemplo `numero__total`;
- una cadena sin comilla doble de cierre;
- un literal de carácter sin cierre o con más de un carácter.

Si la entrada puede separarse en lexemas reconocibles, el analizador léxico retorna dichos lexemas sin evaluar la estructura completa de la instrucción.

4. Pruebas de reconocimiento

4.1. Programa completo

El siguiente programa reúne las principales estructuras definidas para FlowxyRust:

```
constante MAYORIA : entero32 = 18;
sea mutable total : entero32 = 0;
sea mutable notas : [entero32; 3] = [10, 15, 20];

clase Persona {
    sea nombre : cadena;
    sea edad : entero32;
}

implementa Persona {
    fun obtenerEdad() : entero32 {
        retornar este.edad;
    }

    fun mutable cumplirAnios() {
        este.edad++;
    }
}
```

```
fun duplicar(numero : entero32) : entero32 {
    retornar numero * 2;
}

fun saludar(nombre : cadena) {
    mostrar("Hola " + nombre);
    retornar;
}

mainfun() {
    sea mutable persona : Persona;
    persona = Persona { nombre: "Diego", edad: 20 };

    sea mutable edadIngresada : entero32;
    leer(edadIngresada);
    persona.edad = edadIngresada;
    persona.cumplirAnios();

    sea edadActual : entero32 = persona.obtenerEdad();
    sea activo : booleano = verdadero;
    sea inicial : caracter = 'D';
    sea temperatura : flotante32 = -5.5;
    sea cantidad : natural32 = 3;
    sea resultado : entero32 = duplicar(edadActual);

    notas = [12, 16, 18];
    notas[0] = (notas[1] + notas[2]) / 2;

    si (edadActual >= MAYORIA && activo) {
        saludar(persona.nombre);
    } sino {
        mostrar("Registro pendiente");
    }

    sea mutable contador : entero32 = 0;
    mientras (contador < 3) {
        mostrar(notas[contador]);
        contador++;
    }

    para (sea mutable i : entero32 = 0; i < 3; i++) {
        total += notas[i];
    }

    // Mostrar el resultado acumulado.
    mostrar(total);
    mostrar(resultado);
}
```

Este ejemplo permite reconocer palabras reservadas, identificadores, enteros, decimales, cadenas, caracteres, operadores simples y compuestos, delimitadores, accesos mediante punto, vectores y comentarios.

4.2. Ejemplos directos

Como verificación rápida, se muestran diez casos representativos:

Entrada	Reconocimiento
`numero_total`	Identificador, token 1000.
`15.5`	Literal numérico, token 2000.
`persona.edad`	Identificador, delimitador `.`, identificador.
`contador++`	Identificador y operador ++, token 220.
`edad >= 18`	Identificador, operador >=, token 209, y literal numérico.
`A/B`	Identificador, operador `/`, token 204, identificador.
`total /= 2`	Identificador, operador `/=`, token 218, literal numérico.
`// comentario`	Comentario reconocido y descartado.
`"Hola"`	Literal de cadena, token 4000.
`'D'`	Literal de carácter, token 3000.

5. Código fuente

La implementación se separó en archivos pequeños para que cada parte tenga una función clara. El recorrido principal está en `lexer.cpp`; los tokens se representan con `Token`; y los identificadores reconocidos se guardan en una tabla simple.

5.1. Estructura del token

Cada lexema reconocido se guarda junto con su código y una descripción.

```
struct Token {
    int codigo;
    std::string descripcion;
    std::string lexema;
};
```

Por ejemplo, al reconocer `Persona` se puede obtener:

```
1000    Identificador    Persona
```

De esta manera la salida conserva el token asignado, el tipo de componente y el lexema que apareció realmente en el programa.

5.2. Estructura del analizador

La clase `Lexer` contiene las funciones encargadas de reconocer cada tipo de lexema.

```
class Lexer {
private:
    SymbolTable& tablaSimbolos;

    bool esDigito(char c);
    Token reconocerIdentificador(const std::string& entrada, int &i);
    Token reconocerNumero(const std::string& entrada, int &i);
    Token reconocerCadena(const std::string& entrada, int &i);
    Token reconocerCaracter(const std::string& entrada, int &i);
    Token reconocerOperador(const std::string& entrada, int &i);

public:
    Lexer(SymbolTable& tabla);
    std::vector<Token> analizar(const std::string& entrada);
};
```

Cada función corresponde a uno de los patrones explicados anteriormente. La variable `i` indica la posición actual y avanza conforme se consumen los caracteres del lexema.

5.3. Palabras reservadas, operadores y delimitadores

Los lexemas fijos se almacenan en tablas junto con su token.

```
unordered_map<string,int> palabrasReservadas = {
    {"sea",100}, {"constante",101}, {"mutable",102},
```

```
    {"clase",103}, {"implementa",104}, {"este",105},
    {"entero32",107}, {"flotante32",113},
    {"leer",120}, {"mostrar",121},
    {"si",122}, {"sino",123}, {"mientras",124},
    {"para",125}, {"fun",126}, {"retornar",127},
    {"mainfun",128}
};

unordered_map<string,int> operadores = {
    {"-",200}, {"!",201}, {"+",202}, {"*",203},
    {"/",204}, {"%",205}, {"<",206}, {"<=",207},
    {">",208}, {">=",209}, {"==",210}, {"!=",211},
    {"&&",212}, {"||",213}, {"=",214},
    {"+=",215}, {"-=",216}, {"*=",217},
    {"/=",218}, {"%=",219}, {"++",220}, {"--",221},
    {"&",911}, {"|",911}
};

unordered_map<char,int> delimitadores = {
    {'(',300}, {')',301}, {'{',302}, {'}',303},
    {'[',304}, {']',305}, {':',306}, {';',307},
    {'.',308}, {',',309}
};
```

Las palabras reservadas se buscan después de formar una palabra. En operadores, los símbolos & y | solos se consideran error; únicamente se aceptan como && y ||.

5.4. Reconocimiento de identificadores

El identificador utiliza dos estados. El estado 1 representa una forma válida y el estado 2 indica que se acaba de leer _ y todavía se necesita una letra o un dígito.

```
Token Lexer::reconocerIdentificador(const string& entrada, int &i){
    string lexema;
    int estado = 1;

    while(i < entrada.size()){
        char c = entrada[i];

        if(estado == 1){
            if(isalnum(c)){
                lexema += c;
                i++;
            }
            else if(c == '_'){
                estado = 2;
                lexema += c;
                i++;
            }
            else{
                break;
            }
        }
        else if(estado == 2){
```

```
            if(isalnum(c)){
                estado = 1;
                lexema += c;
                i++;
            }
            else{
                break;
            }
        }
    }

    if(estado == 2 ||
       (i < entrada.size() && entrada[i] == '_')){
        while(i < entrada.size() &&
              (isalnum(entrada[i]) || entrada[i] == '_')){
            lexema += entrada[i];
            i++;
        }
        return {911, "Error lexico", lexema};
    }

    if(palabrasReservadas.count(lexema)){
        return {
            palabrasReservadas[lexema],
            "Palabra reservada",
            lexema
        };
    }

    tablaSimbolos.insertar(lexema);
    return {1000, "Identificador", lexema};
}
```

Por ejemplo, `numero_total` vuelve al estado válido después del guion bajo y se reconoce como identificador. En cambio, `numero_` termina en el estado intermedio y se devuelve el código `911`. Al finalizar una palabra también se comprueba si el lexema pertenece a las palabras reservadas.

5.5. Reconocimiento de números

El número también utiliza estados para distinguir la parte entera del posible decimal.

```
Token Lexer::reconocerNumero(const string& entrada, int &i){
    string lexema;
    int estado = 1;

    while(i < entrada.size()){
        char c = entrada[i];

        if(estado == 1){
            if(isdigit(c)){
                lexema += c;
                i++;
            }
            else if(c == '.'){
```

```
                estado = 2;
                lexema += c;
                i++;
            }
            else{
                break;
            }
        }
        else if(estado == 2){
            if(isdigit(c)){
                estado = 3;
                lexema += c;
                i++;
            }
            else{
                break;
            }
        }
        else if(estado == 3){
            if(isdigit(c)){
                lexema += c;
                i++;
            }
            else if(c == '.'){
                estado = 99;
                lexema += c;
                i++;
                break;
            }
            else{
                break;
            }
        }
    }

    if(estado == 2 || estado == 99){
        while(i < entrada.size() &&
              (isalnum(entrada[i]) || entrada[i] == '.')){
            lexema += entrada[i];
            i++;
        }
        return {911, "Error lexico", lexema};
    }

    return {2000, "Literal numerico", lexema};
}
```

El estado 1 reconoce la parte entera, el estado 2 indica que se leyó el punto y se espera un dígito, y el estado 3 corresponde a un decimal válido. Así, `20` y `15.5` retornan `2000`, mientras que `15.` o `15.5.2` producen error léxico.

5.6. Reconocimiento de cadenas y caracteres

Para una cadena se consume desde la primera comilla doble hasta encontrar la comilla de cierre.

```
Token Lexer::reconocerCadena(const string& entrada,int &i){
    string lexema;
    lexema += entrada[i];
    i++;

    while(i < entrada.size() && entrada[i] != '"'){
        lexema += entrada[i];
        i++;
    }

    if(i >= entrada.size()){
        return {911, "Error lexico", lexema};
    }

    lexema += '"';
    i++;

    return {4000, "Literal de cadena", lexema};
}
```

Si la entrada termina antes de encontrar la segunda comilla, la cadena queda incompleta y se retorna `911`.

El literal de carácter exige una comilla inicial, un carácter y una comilla final.

```
if (i < entrada.size() && entrada[i] != '\'') {
    lexema += entrada[i++];

    if (i < entrada.size() && entrada[i] == '\'') {
        lexema += entrada[i++];
        return {3000, "Literal de caracter", lexema};
    }
}

return {911, "Error lexico", lexema};
```

Por ello, `'D'` es válido, mientras que un carácter sin cierre o con una forma diferente se reporta como error léxico.

5.7. Reconocimiento de operadores

Para los operadores primero se forma el símbolo de un carácter y luego se comprueba si junto con el siguiente forma un operador compuesto.

```
Token Lexer::reconocerOperador(const string& entrada,int &i){
    string lexema;
    lexema += entrada[i];

    if(i + 1 < entrada.size()){
        string doble = lexema + entrada[i + 1];
```

```
        if(operadores.count(doble)){
            lexema = doble;
            i += 2;
            return {
                operadores[lexema],
                "Operador",
                lexema
            };
        }
    }

    i++;

    int tokenCode = operadores[lexema];

    if(tokenCode == 911)
        return {911, "Error lexico", lexema};

    return {tokenCode, "Operador", lexema};
}
```

Con este procedimiento, al leer > se comprueba si continúa con =; lo mismo ocurre con ++, +=, /= y los demás operadores compuestos.

5.8. Recorrido principal del scanner

La función analizar recorre toda la entrada. El primer carácter de cada lexema decide qué función de reconocimiento debe utilizarse.

```
vector<Token> Lexer::analizar(const string& entrada){
    vector<Token> tokens;
    int i = 0;

    while(i < entrada.size()){
        if(isspace(entrada[i])){
            i++;
            continue;
        }

        if(entrada[i] == '/' &&
           i + 1 < entrada.size() &&
           entrada[i + 1] == '/'){
            while(i < entrada.size() &&
                  entrada[i] != '\n'){
                i++;
            }
            continue;
        }

        if(entrada[i] == '"'){
            tokens.push_back(reconocerCadena(entrada,i));
            continue;
        }
```

```
        if(entrada[i] == '\''){
            tokens.push_back(reconocerCaracter(entrada,i));
            continue;
        }

        if(isalpha(entrada[i])){
            tokens.push_back(
                reconocerIdentificador(entrada,i)
            );
            continue;
        }

        if(esDigito(entrada[i])){
            tokens.push_back(reconocerNumero(entrada,i));
            continue;
        }

        string temp(1,entrada[i]);

        if(operadores.count(temp)){
            tokens.push_back(reconocerOperador(entrada,i));
            continue;
        }

        if(delimitadores.count(entrada[i])){
            tokens.push_back({
                delimitadores[entrada[i]],
                "Delimitador",
                temp
            });
            i++;
            continue;
        }

        tokens.push_back({911, "Error lexico", temp});
        i++;
    }

    return tokens;
}
```

Aquí se observa directamente el funcionamiento general del autómata: se ignoran espacios, se descartan comentarios, y luego se elige el camino correspondiente para cadenas, caracteres, identificadores, números, operadores o delimitadores. Si el carácter no pertenece a ningún caso, se devuelve 911.

5.9. Tabla de identificadores

Los identificadores se guardan en un unordered_set, por lo que un mismo lexema no se inserta dos veces.

```
void SymbolTable::insertar(const std::string& lexema) {
    tabla.insert(lexema);
}
```

```
const std::unordered_set<std::string>&
SymbolTable::obtener_tabla() const {
    return tabla;
}
```

Esta tabla se utiliza solamente para conservar los identificadores reconocidos sin repetirlos.

5.10. Entrada y salida del programa

El programa puede recibir el código directamente por consola o leerlo desde un archivo. Luego llama al lexer y muestra los tokens obtenidos.

```
SymbolTable tablaSimbolos;
Lexer lexer(tablaSimbolos);
string codigo = "";

// ...

vector<Token> resultado = lexer.analizar(codigo);

cout << "TOKEN\tDESCRIPCION\t\tLEXEMA\n";

for(auto t : resultado){
    cout << t.codigo << "\t"
         << t.descripcion << "\t\t"
         << t.lexema << "\n";
}
```

De esta forma la salida muestra el token, la descripción y el lexema de cada componente reconocido en el mismo orden en que aparece en el programa fuente.